

Week 9 Student Handbook

[Progress Video + Verification Lab](#) | [Practice Session 8](#) | [Trustworthy AI 51.516](#)

Table of Contents

Use Ctrl+Click on a heading to jump. Update field with F9 in Word.

Table of Contents	2
1. What Week 9 is.....	4
Session timing (in class)	4
What you produce this week	4
2. Pair roles.....	5
3. Property Types - choose one for your lab.....	6
4. Property Type A - Local Robustness (most common)	7
5. Property Type B - Safety Threshold	8
6. Property Type C - Monotonicity	9
7. Property Type D - Bounded Difference (baseline vs defended)	10
8. Vocabulary	11
9. Progress video preparation.....	12
Preparation timeline	12
10. 5-slide progress deck.....	13
11. How to record on Microsoft Teams.....	14
Setup.....	14
Recording.....	14
Retrieve and submit	14
Structure of the 5-minute video	14
If a take goes wrong.....	14
12. Anticipate the questions I'll ask in the video review	15
13. Verification lab - Choose your tool.....	16
14. Verification lab - Step 1 - Install Marabou.....	17
15. Verification lab - Step 2 - Export your model to ONNX.....	18
16. Verification lab - Step 3 - State your property in plain English.....	19
17. Verification lab - Step 4 - Encode the property	20
18. Verification lab - Step 5 - Run and interpret.....	21
19. What good looks like.....	22
EXCELLENT verification report.....	22
WEAK verification report	22
20. Common mistakes to avoid	23
21. Deliverable Template - verification report (1 to 2 pages).....	24
21.1 Property statement.....	24
21.2 Tool and configuration.....	24

21.3 Input region	24
21.4 Output constraint.....	24
21.5 Result	24
21.6 Interpretation (3 to 5 sentences)	25
21.7 Reflection	25
22. Rubric (verification report, marked out of 100 - contributes to 5% Progress Presentation grade)	26
Grade bands	26
23. Submission checklist	27

1. What Week 9 is

Week 9 has two pieces of work: a Progress Video and a Verification Lab. The Progress Video is a 5-minute recording your pair (team of 2) does on Microsoft Teams, showing the current state of your course project (attack, defense, and demo). The Verification Lab is a hands-on exercise you complete on your own time (roughly 3 to 4 hours), running Marabou or ERAN against ONE verification property on your project model.

Each piece has a specific pedagogical intent. The Progress Video trains you to walk a stakeholder through a project succinctly (a real workplace skill; think design reviews). The Verification Lab introduces you to a discipline (formal verification) that regulated industries increasingly demand, in a way that empirical testing cannot substitute for.

Session timing (in class)

0 to 30 min	Lecture content (verification concepts, real-world cases, tool landscape)
30 to 60 min	Guest lecture: Rishabh Rajan, WalledAI (industry perspective on verification and behavioural evaluation)
60 to 90 min	Lab briefing, Q&A, book Week 11 consultation slot

What you produce this week

- Progress video (MP4, recorded on Microsoft Teams, 5 minutes max), submitted 7 days after class
- Progress deck (5 slides, PDF), the deck shown in your video
- Verification report (1 to 2 pages, PDF), submitted with the video
- ONNX model export and property file committed to your project repository

2. Pair roles

Both partners appear on camera in the progress video. Both partners write the verification report. The role split below is a starting frame for who OWNS which piece of the prep.

Partner A: Narrator + Property Owner	Owns the 5-slide deck narrative and drives the recorded demo section. Also drafts the verification property (Section 6) before recording so the pair has something concrete to walk through.
Partner B: Demo Operator + Interpretation Owner	Owns running the code during the recorded demo (screen share). Also owns the interpretation section of the verification report, connecting the SAT/UNSAT/UNKNOWN result to what it means for the project's threat model.

Both partners must appear on camera in the video AND narrate at least two of the five slides each. Split ownership of tasks; share ownership of understanding.

3. Property Types - choose one for your lab

Formal verification proves ONE testable claim about your network over ONE input region. In this lab you pick one of the four property types below and defend your choice. Full detail for each type is in Sections 4 to 7.

Type A: Local Robustness	Prove the network does not flip a prediction within a small perturbation ball around a benign input. Most common. Best for classifiers.
Type B: Safety Threshold	Prove that when a sensor reading is in a defined safe range, an output (alarm, control signal) stays within a defined bound. Best for control systems and threshold detectors.
Type C: Monotonicity	Prove that when one feature increases with others held constant, the output does not decrease. Best for credit scoring, pricing, and any model with monotonic domain assumptions.
Type D: Bounded Difference	Prove that a baseline network and a defended network stay within delta of each other on a defined region. Best for regression tests when you have adversarially trained a defended variant.

The lab requires ONE property type verified end to end. If you cannot make Type A work on your network within timeout, switching to a smaller region or Type B or C is legitimate; document the switch in your report.

4. Property Type A - Local Robustness (most common)

Prove: for a chosen benign input x_0 , for all inputs x with L -infinity distance from x_0 at most ϵ , the network's predicted class stays the same.

Best for

- Image, text, or tabular classifiers
- Small-to-medium networks (under 5,000 ReLU neurons for Marabou, more with alpha-beta-CROWN on GPU)
- Threat models where the attacker perturbs the input slightly, staying within a well-defined budget

Property template

WORKED EXAMPLE - Local robustness template

Plain English: For a chosen benign 'cat' image x_0 from the test set, any input within L -infinity distance 0.008 of x_0 continues to be classified as 'cat'.

Formal: For all x in $[0, 1]^{3 \times 32 \times 32}$ with $\max_i |x[i] - x_0[i]| \leq 0.008$, model output y satisfies $y[\text{cat}] > y[j] + 0.0001$ for all $j \neq \text{cat}$.

Guiding questions (Type A)

- Which benign input do you certify against? Is it a typical example or an edge case?
- What epsilon matches your threat model? 0.03 is common for MNIST; 0.008 is common for CIFAR-10 with adversarial training.
- If your model fails at a large epsilon, at what smaller epsilon does it succeed? Report both.

5. Property Type B - Safety Threshold

Prove: for a defined input region (typically ranges on physical sensor readings), the output stays within a defined bound (alarm below a threshold, control signal within a safe range).

Best for

- Control systems trained with neural networks (drone flight, robotic arm)
- Anomaly detectors where a false negative in a known safe region is unacceptable
- Any regulated domain where you can name a safety envelope numerically

Property template

WORKED EXAMPLE - Safety threshold template

Plain English: For all sensor readings where temperature is between 10 and 30 degrees Celsius and pressure is between 90 and 110 kPa, the neural anomaly detector's alarm signal stays below 0.5.

Formal: For all x with $10 \leq x[\text{temp}] \leq 30$ and $90 \leq x[\text{pressure}] \leq 110$, output $y[\text{alarm}] < 0.5$.

Guiding questions (Type B)

- What is your safety envelope numerically? Get precise bounds, not fuzzy language.
- If the property fails (SAT), what is the counterexample telling you? Is the model correct and the envelope wrong, or is the model wrong?

6. Property Type C - Monotonicity

Prove: for a specified feature, if that feature increases with all other features held constant, the output does not decrease (or does not increase, depending on domain assumption).

Best for

- Credit scoring (higher income should not decrease approval probability)
- Pricing models (higher demand should not decrease predicted price)
- Insurance underwriting
- Any regulated domain where a monotonic assumption is a legal or ethical requirement

Property template

WORKED EXAMPLE - Monotonicity template

Plain English: If applicant income increases with all other features held constant, approval probability does not decrease.

Formal: For all x_1, x_2 with $x_2[\text{income}] > x_1[\text{income}]$ and $x_2[j] = x_1[j]$ for all other j , model output $y[\text{approve}]$ evaluated on x_2 is $\geq y[\text{approve}]$ evaluated on x_1 .

Guiding questions (Type C)

- Which feature and which direction is required by domain assumption?
- Over what range of the feature must monotonicity hold? Realistic ranges are usually enough (income from 0 to 500k, not from 0 to infinity).
- If the property fails, what pair of inputs violates it? Show them side by side in the report.

7. Property Type D - Bounded Difference (baseline vs defended)

Prove: two networks (typically your baseline model and your adversarially trained defended model) produce outputs within delta of each other on a defined benign region.

Best for

- Comparing a baseline and a defended variant of the same model
- Certifying that a fine-tuning step did not change the model's behaviour on the trust region
- Regression test after a model update

Property template

WORKED EXAMPLE - Bounded difference template

Plain English: On the benign test region, the baseline model and the adversarially trained defended model produce outputs within 0.05 of each other on the target class.

Formal: For all x in `benign_region`, $|y_{\text{baseline}}[\text{target}] - y_{\text{defended}}[\text{target}]| \leq 0.05$.

Guiding questions (Type D)

- What is the benign region? Define it precisely.
- What delta counts as 'the same behaviour' for your use case?
- If the property fails, does that mean the defence changed behaviour more than acceptable, or is your delta too tight?

8. Vocabulary

VOCAB - Formal verification

A mathematical procedure that proves whether a property holds for every possible input in a defined region. Unlike testing (checks a finite number of inputs), verification covers the whole region at once.

VOCAB - Property

A testable claim of the shape 'in this input region, the network always outputs this constraint.' Property types A through D are covered in Sections 4 to 7.

VOCAB - Input region

The set of inputs you want the property to hold over. Usually defined by bounds: an L-infinity ball around a specific point, per-feature ranges, or a subspace of the input space.

VOCAB - Sound verifier

When it says YES the property truly holds. May sometimes fail to conclude (UNKNOWN or timeout), but never falsely claims safety. Marabou is sound.

VOCAB - Complete verifier

When the property holds, will always say YES. Never returns UNKNOWN when the answer is YES. Complete verifiers are rare in practice for medium networks.

VOCAB - Counterexample

A specific input inside your defined region that violates the property. If a verifier returns SAT, it prints one. Study it; it is a genuine finding.

VOCAB - ReLU

Rectified Linear Unit, the function $\max(0, x)$. Each ReLU introduces a case split (active or inactive) that verifiers must reason about. Network size drives verification difficulty.

VOCAB - ONNX

Open Neural Network Exchange, a common serialization format that verifiers read. Both Marabou and ERAN require ONNX input.

VOCAB - Safety case

A structured written argument that a system is acceptably safe for a stated operational domain, backed by evidence. Waymo uses this approach for autonomous driving; Tesla does not.

9. Progress video preparation

Preparation timeline

1 week before	Choose your property type from Section 3. Start the verification lab installation in parallel.
3 days before	Rehearse the 5-slide deck out loud. Time it. First take is usually 7 minutes; you need to get to 5.
2 days before	Do a dry-run recording between the two of you. Watch it back. Note the bits that need tightening.
1 day before	Record the final take. Watch it back. Re-record if needed. You have unlimited takes.
Day of submission	Upload the MP4 plus deck and verification report to the course portal.

10. 5-slide progress deck

1. Slide 1 - Title: project name, pair, one-line use case
2. Slide 2 - Threat model: attack you defend against in one sentence
3. Slide 3 - System under test: model architecture, dataset, evaluation harness
4. Slide 4 - Progress: clean accuracy, attack success rate, defense effect. ONE chart, not bullets.
5. Slide 5 - Verification plan and top open risk

WORKED EXAMPLE - Sample slide 4 speech (fraud detection project)

'Baseline XGBoost achieves F1 0.83 on the minority class. Under an L-infinity feature perturbation attack at epsilon 0.05, F1 collapses to 0.44. Adversarial training augmentation restores F1 to 0.74, with a 2-point loss on clean data. That is our headline number. The chart shows the three metrics side by side.'

WORKSHEET - Progress deck skeleton

Fill in ONE sentence per slide before recording. This is your script.

Slide 1 (title): _____

Slide 2 (threat model): _____

Slide 3 (system under test): _____

Slide 4 (progress metrics): _____

Slide 5 (verification plan + risk): _____

11. How to record on Microsoft Teams

Setup

- Both partners on the Teams call (Meet Now works fine). Different locations is fine.
- Both partners have cameras on for the entire recording.
- One partner shares screen for the deck and demo.

Recording

- Hit the Record button in the top toolbar (three-dot menu > Record).
- Say the pair name and project name in the first 5 seconds so the video is identifiable.
- Walk through the 5 slides, both partners narrating at least 2 slides each.
- For the demo section: share screen with the code and results visible, run it, narrate.
- Stop recording. Teams saves to OneDrive automatically.

Retrieve and submit

- Download the MP4 from OneDrive (Files > Recordings).
- Rename: W9_Video_PairName.mp4
- Upload to the course portal along with the deck PDF and verification report PDF.

Structure of the 5-minute video

- 0:00 to 0:30 - Title slide + pair introductions (both faces on camera)
- 0:30 to 1:30 - Threat model + system under test (slides 2 and 3)
- 1:30 to 3:30 - Progress slide (slide 4) + demo: input, attack, defense, human decision point
- 3:30 to 4:30 - Verification plan + top open risk (slide 5)
- 4:30 to 5:00 - Wrap: what you're most confident in, what you're least confident in

If a take goes wrong

Re-record. You have unlimited takes; nobody sees the outtakes. This is a huge advantage over live presenting.

12. Anticipate the questions I'll ask in the video review

I'll watch each video and file feedback. Address these in the video so I don't have to ask:

- Why this specific threat model and not a broader one?
- Why this defense over the obvious alternative?
- What is your worst-case metric under attack, and how confident are you in it?
- What are you not yet defending against?
- What one thing would change if you had another two weeks?

If the video is silent on any of these, expect a follow-up question in writing.

13. Verification lab - Choose your tool

Marabou	SMT-based sound verifier. If it says UNSAT the property truly holds. If SAT you get a counterexample. Best for networks under ~5,000 neurons. CPU.
ERAN	Abstract-interpretation verifier using zonotope and polyhedra domains. Faster on medium networks. May return UNKNOWN when bounds are too loose. CPU.
alpha-beta-CROWN	Learned bound tightening, state of the art in VNN-COMP. Use only if you have GPU access. Not required for the lab.

WORKED EXAMPLE - Tool choice example

Project: a 3-layer MLP for spam classification (~15,000 neurons). Property Type A local robustness at epsilon 0.05.

Reasoning: 15,000 neurons is above Marabou's comfortable range but within ERAN's zonotope domain range. Pick ERAN with the deeppoly domain. If UNKNOWN, retry at a smaller epsilon (0.02) or switch to a smaller subnetwork.

14. Verification lab - Step 1 - Install Marabou

Marabou is a sound SMT-based verifier. If it says UNSAT, the property holds; if SAT, it prints a counterexample; if UNKNOWN, it could not decide within the timeout.

Install from source (pip package is often out of date):

```
# Dependencies: g++ 7+, CMake 3.2+, Python 3.7+
git clone https://github.com/NeuralNetworkVerification/Marabou.git
cd Marabou && mkdir build && cd build
cmake .. -DBUILD_PYTHON=ON
make -j4    # 5-10 minutes
./Marabou --version
```

WORKED EXAMPLE - Expected outcome and troubleshooting

Success: version line prints, e.g. 'Marabou 2.0.0'.

make fails with 'error: unknown type name' - install build-essential (Ubuntu) or Xcode command-line tools (Mac).

make fails on CMake - check cmake --version >= 3.2.

Python module not found after install - export PYTHONPATH to include the Marabou install directory.

15. Verification lab - Step 2 - Export your model to ONNX

Verifiers do not read PyTorch or TensorFlow objects. They read ONNX. Exporting also forces you to freeze the input shape and remove any Python-side control flow.

```
import torch

model = torch.load('model.pth').eval()

dummy_input = torch.randn(1, 784)    # match your real input shape

torch.onnx.export(model, dummy_input, 'network.onnx',
                  input_names=['x'], output_names=['logits'], opset_version=12)
```

WORKED EXAMPLE - Expected outcome and sanity check

Success: network.onnx file created, roughly the size in KB of the number of parameters.

Sanity check via onnxruntime: run forward pass on the ONNX file and confirm it matches PyTorch on a test sample within $1e-5$ max difference.

If max diff $> 1e-5$: an operator did not round-trip. Common culprits: custom layers, Python if-statements, layer-dependent dropout.

Fix: rewrite the problematic layer in pure PyTorch tensor operations and re-export.

16. Verification lab - Step 3 - State your property in plain English

Before you write anything formal, write the property in plain English. If you cannot say it in plain English in ONE sentence, you do not yet understand the property. Pick a property type from Section 3 (A local robustness, B safety threshold, C monotonicity, D bounded difference).

WORKSHEET - Property statement (fill in for your project)

Write your property in plain English first, then in formal form with input bounds and output constraint.

Property type (A / B / C / D): _____

Plain English (one sentence): _____

Formal input bounds: _____

Formal output constraint: _____

17. Verification lab - Step 4 - Encode the property

Marabou takes a property file listing input bounds then output constraints. Example for Type A local robustness on a 784-input, 10-output MNIST model, target class 7:

```
# property.txt  
x0 >= 0.5  
x0 <= 0.6  
x1 >= 0.4  
x1 <= 0.5  
# ... (all 784 input bounds: [x0_i - 0.03, x0_i + 0.03])  
  
+y7 -y0 >= 0.0001  
+y7 -y1 >= 0.0001  
# ... (each non-target class must be strictly less than y7)
```

WORKED EXAMPLE - Encoding tips

For Type A: script the input bounds programmatically from a benign x_0 numpy array. Do not hand-type 784 lines.

For Type B: bounds map directly to your safety envelope; use the numeric bounds from your domain spec.

For Type C: encode monotonicity as a two-input comparison; you may need to run TWO queries and combine.

For Type D: use Marabou's dual-network mode or run two queries on each network and post-process.

18. Verification lab - Step 5 - Run and interpret

Run:

```
./Marabou network.onnx property.txt --timeout=1800 > result.log 2>&1
```

Interpret:

UNSAT	Property holds over the entire region. Certificate at this epsilon for this input.
SAT	Counterexample. Marabou prints an input; run it through the model to confirm.
UNKNOWN or TIMEOUT	Verifier could not decide in your budget. Shrink the region, shrink the network, or switch tools.

WORKED EXAMPLE - MNIST 2-layer MLP verified at epsilon 0.03 (worked example)

Model: 784 -> 64 (ReLU) -> 10 logits, ~50,000 parameters. Exported as network.onnx.

Property: for image index 42 (a '7'), for all inputs x with $\max_i |x[i] - x_0[i]| \leq 0.03$, $y[7] > y[j] + 0.0001$ for all $j \neq 7$.

Tool: Marabou 2.0, single CPU. Encoded 784 input bounds and 9 output inequalities.

Result: UNSAT after 41.2 seconds with 3,124 ReLU splits. Certifies robustness at this image at epsilon 0.03.

Sanity check: rerun at epsilon 0.10. Marabou returns SAT and prints a counterexample x^* with max change 0.098 on 12 pixels; the network predicts class 3 there. This confirms the tool is behaving correctly and 0.03 is a real threshold.

WORKSHEET - Result and interpretation (fill in for your run)

Your actual numbers, not the sample numbers above.

Result (UNSAT / SAT / UNKNOWN): _____

Wall clock time (seconds): _____

If SAT, key features of counterexample: _____

Interpretation in one sentence: _____

19. What good looks like

EXCELLENT verification report

'For a benign CIFAR-10 test image (index 42, class cat), we verified with alpha-beta-CROWN that no L-infinity perturbation up to epsilon 0.008 changes the predicted class. Runtime 4.7 seconds. UNSAT with zero branch-and-bound iterations because CROWN bounds were already tight enough. To connect this to our threat model, we re-ran the same certification across 200 test images and computed a certified accuracy of 68 percent at epsilon 0.008. Our empirical PGD accuracy at the same epsilon is 71 percent, so certified accuracy is a slightly conservative lower bound as expected. We interpret this as strong evidence that our adversarially trained defence generalises beyond the specific attacks we tested empirically.'

WEAK verification report

'We ran Marabou on our model. It said UNSAT. The model is verified.'

20. Common mistakes to avoid

- Property so large the verifier will time out. Start small; grow gradually.
- Aspirational property. 'The model is fair' is NOT verifiable. 'The output for one specific input does not flip within epsilon 0.03' IS.
- Reporting only 'verified' without saying at what epsilon, on which input, with which tool. Certificates without provenance are worthless.
- Treating UNKNOWN as failure. UNKNOWN means the tool could not decide; report it honestly and try a tighter setting.
- Verifying the WRONG property. Boeing 737 MAX MCAS was tested against a specification that allowed a single AoA sensor input to repeatedly push the nose down. 346 people died from a spec-versus-safety gap. When you write your property, ask: is this actually what I want to guarantee?

21. Deliverable Template - verification report (1 to 2 pages)

Fill each section in your own words. Amber blocks show a strong sample using an adversarially trained CIFAR-10 ResNet-8 with a Type A property. Model on the standard without copying.

21.1 Property statement

Prompt: state your property in plain English, then in formal form. Name the property type (A/B/C/D).

WORKED EXAMPLE - Sample (Type A, CIFAR-10)

Type: A - Local Robustness.

Plain English: For a chosen benign 'cat' image x_0 from the CIFAR-10 test set, any input within L-infinity distance 0.008 of x_0 continues to be classified as 'cat' by our adversarially trained ResNet-8.

Formal: For all x in $[0, 1]^{3 \times 32 \times 32}$ with $\max_i |x[i] - x_0[i]| \leq 0.008$, $y[\text{cat}] > y[j] + 0.0001$ for all $j \neq \text{cat}$.

21.2 Tool and configuration

WORKED EXAMPLE - Sample

Tool: alpha-beta-CROWN v0.3.1 via the auto_LiRPA library, incomplete verifier mode.

Hardware: single NVIDIA T4 GPU, 16 GB RAM.

Config: branch-and-bound with 30-second timeout per property, batch size 8, initial bounds from CROWN-IBP.

21.3 Input region

WORKED EXAMPLE - Sample

Per-pixel L-infinity bounds of 0.008 around benign image x_0 (CIFAR-10 test set index 42). All pixels clamped to $[0, 1]$.

21.4 Output constraint

WORKED EXAMPLE - Sample

$y[\text{cat}] - y[j] > 0.0001$ for all j in {airplane, automobile, bird, deer, dog, frog, horse, ship, truck}.

21.5 Result

WORKED EXAMPLE - Sample

Verified (all 9 constraints proved) in 4.7 seconds on the GPU. No branch-and-bound required beyond the initial CROWN pass.

21.6 Interpretation (3 to 5 sentences)

WORKED EXAMPLE - Sample

The result certifies that for this specific image, no L_∞ perturbation up to 0.008 can cause a misclassification. This is a stronger claim than an empirical PGD evaluation which only shows no attack succeeded within a finite compute budget. The guarantee is pointwise: certification at 100 test images does not generalize to unseen images. In practice we would repeat this across a benchmark of at least 200 images to compute a certified accuracy percentage and compare against empirical PGD accuracy at the same epsilon.

21.7 Reflection

WORKED EXAMPLE - Sample

Verification is stronger because it covers the entire epsilon-ball, not just tested points. It is weaker because it does not tell us anything about larger perturbations, semantic changes (rotation, brightness), or distribution shift over time. Our project will report certified accuracy at epsilon 0.008 alongside empirical PGD accuracy at epsilon 0.031 to give a fuller picture in the final report. If our adversarial training pipeline changes, we will re-run this certification to catch regressions.

22. Rubric (verification report, marked out of 100 - contributes to 5% Progress Presentation grade)

Property specification (20)	Property type named; property stated in plain English AND formal form; input bounds precise.
Tool and configuration (10)	Tool named, version, hardware, non-default settings documented.
Result reporting (15)	UNSAT/SAT/UNKNOWN, runtime, and (if SAT) counterexample.
Interpretation (20)	Three to five sentences connecting the result to the threat model. Distinguishes what verification gives vs empirical testing.
Reflection (15)	Names limits of what was verified. Names next step if the certification would need to be extended.
Writing and rigor (20)	Concise, honest about UNKNOWN, no aspirational claims. 2 pages maximum.

Grade bands

Exemplary (85+)	Every section precise. Interpretation genuinely connects to project threat model. Reflection honest about limits.
Proficient (70-84)	Most sections precise. Interpretation mostly right; reflection acknowledges some limits.
Developing (55-69)	Property loose; interpretation confuses verification with empirical testing; reflection weak.
Below (0-54)	Aspirational property; UNKNOWN treated as failure; report shorter than 1 page or exceeds 2.

23. Submission checklist

- W9_Video_PairName.mp4 - 5-minute progress video, recorded on Teams, both partners on camera
- W9_Deck_PairName.pdf - the 5-slide deck shown in the video
- W9_Verification_PairName.pdf - 1 to 2 page verification report using the template in Section 21
- ONNX file and property file committed to your project repo
- Every claim in the report has an evidence anchor (tool version, benign image index, epsilon, etc.)
- Reflection distinguishes verification from empirical testing
- Both partners' names on the report cover line and both narrate in the video
- Upload all four files (video, deck, report, repo link) to the course portal within 7 days of class